

Drive-Saver Copilot

Technical Documentation

Agentic prescriptive maintenance for variable frequency drives and electric motors

ABB Accelerator 2026, Theme 1: Agentic Predictive Maintenance Studio

Version 1.0, September 2026

Live demo: <https://drive-saver-copilot-138499493256.us-central1.run.app>

Source code: <https://github.com/riajha-usc/drive-saver-copilot>

Contents

- | | |
|--------------------------|---------------------------------|
| 1. Overview | 8. API reference |
| 2. Solution architecture | 9. Dashboard |
| 3. Technologies used | 10. Deployment |
| 4. Data pipeline | 11. Testing and validation |
| 5. Failure prediction | 12. Setup instructions |
| 6. Explainability | 13. Limitations and future work |
| 7. Prescriptive agent | 14. Attribution |

1. Overview

Problem

Predictive maintenance systems typically stop at a warning, for example "this motor will fail in 20 hours". The operator must then decide what to do, usually under time pressure, and the result is often an unplanned stop during a production run, which is the costliest outcome for a plant.

Solution

Drive-Saver Copilot is prescriptive. For a motor driven by a variable frequency drive (VFD), it determines the smallest change to the drive's torque and speed setpoints, or a maintenance action, that moves the predicted failure past the next planned maintenance window. Every prescription states the hours of life gained, the change in failure risk, the output given up, and the estimated cost avoided.

The system combines three capabilities:

1. **Prediction.** Gradient boosted models estimate overall failure risk and the risk of each failure mode. Each model is selected automatically from several algorithms on validation data, with every candidate tracked in MLflow.
2. **Explanation.** SHAP attributions identify the signals driving the risk and the control lever that moves them.
3. **Prescription.** A LangGraph agent simulates candidate adjustments, rejects any that would breach another physical limit, and selects the smallest change that reaches the maintenance window.

Key results

Measure	Result
Critical assets given a safe, working prescription	303 of 314 (96 percent)
Prescriptions that introduce a new failure	0
Time per recommendation	about 30 ms
Overall failure model, test PR AUC	0.890
Automated tests	126, all passing

2. Solution architecture

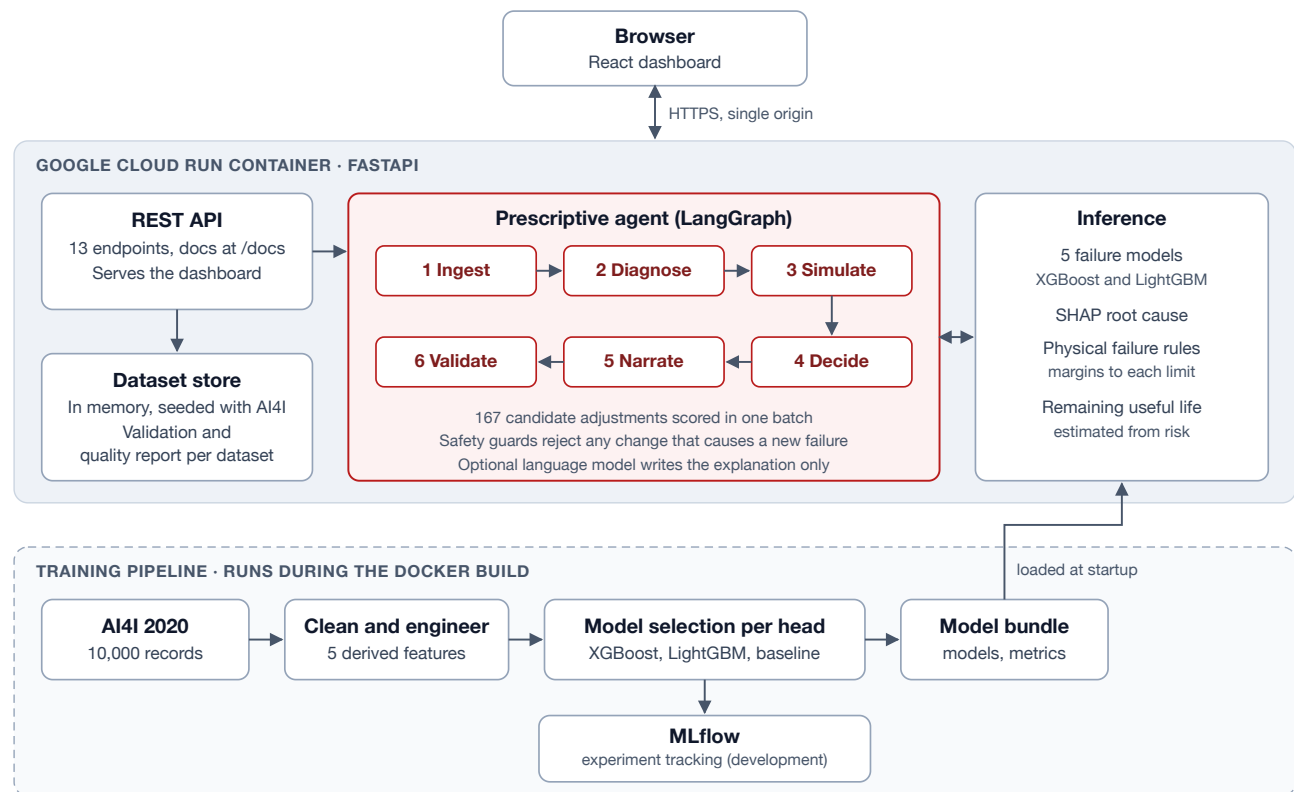


Figure 1. Solution architecture

The application is a single container. FastAPI serves both the REST API and the compiled React dashboard from the same origin, which removes the need for cross origin configuration and gives one URL for the whole product.

Components

Component	Location	Responsibility
Physics rules	<code>backend/physics.py</code>	Failure conditions and signed margins to each physical limit
Data pipeline	<code>backend/ml/dataset.py</code>	Ingestion, validation, feature engineering
Quality report	<code>backend/ml/quality.py</code>	Dataset profiling and fitness verdict
Model training	<code>backend/ml/train.py</code>	Per head model selection, MLflow tracking
Inference	<code>backend/ml/predict.py</code>	Risk, remaining useful life, root cause
Explainability	<code>backend/ml/explain.py</code>	SHAP attributions and control levers
Prescriptive agent	<code>backend/agent/</code>	Counterfactual search, decision, narration, output schema
API	<code>backend/api/</code>	REST endpoints, dataset store, static dashboard serving

Component	Location	Responsibility
Dashboard	frontend/	React interface

Request lifecycle

For a recommendation request on one asset:

1. The API resolves the asset's operating point from the selected dataset.
2. The agent diagnoses it: failure probability, risk band, likely failure mode, margins to each physical limit, and SHAP root cause.
3. It generates 167 candidate adjustments, scores them in a single batched model call, and discards any that breach a new limit.
4. It selects the smallest candidate that clears every breached limit and reaches the maintenance window, or recommends stopping if none can.
5. It writes a plain language explanation, validates the full payload against a strict schema, and returns it with a record of each reasoning step.

3. Technologies used

Technology	Version	Role
Python	3.12 (image), 3.12+ supported	Backend language
XGBoost	3.4	Gradient boosted failure models
LightGBM	4.7	Gradient boosted failure models
scikit-learn	1.9	Baseline model, data splits, metrics
pandas, NumPy	3.0, 2.5	Data processing
SHAP	0.52	Feature attribution
MLflow	3.16	Experiment tracking (development)
LangGraph	1.2	Agent orchestration
Anthropic Claude (optional)	via langchain-anthropic	Explanation wording
FastAPI, Pydantic	0.141, 2.13	REST API and data contracts
React, Vite, Recharts	19, 7, 3	Dashboard
Docker	multi stage build	Packaging
Google Cloud Run	managed	Hosting

4. Data pipeline

Dataset

The system is trained on the **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository (10,000 records, licensed CC BY 4.0). Each record is an operating point with six signals and failure labels.

Signal	Unit	Relevance
Product quality class	L, M, H	Sets the overstrain limit
Air temperature	K	Ambient condition
Process temperature	K	Thermal load
Rotational speed	rpm	Drive output, a control lever
Torque	Nm	Drive load, a control lever
Tool wear	min	Maintenance state

Labels cover overall machine failure and five modes: tool wear (TWF), heat dissipation (HDF), power (PWF), overstrain (OSF) and random failure (RNF). RNF represents 0.1 percent injected noise with no physical cause and is excluded from modelling.

Validation

Ingestion accepts both column spellings in which the dataset is distributed (with or without bracketed units). Rows are rejected, and counted by reason, when the product class is unknown, a reading is missing or non numeric, speed is zero or negative, torque or tool wear is negative, or the row duplicates another. A file missing a required column is refused with a message naming the column.

Feature engineering

Feature	Definition	Purpose
<code>temp_diff</code>	Process minus air temperature	Heat dissipation driver
<code>power_w</code>	Torque multiplied by angular speed	Power failure driver
<code>wear_torque</code>	Tool wear multiplied by torque	Overstrain driver
<code>osf_utilisation</code>	Wear torque divided by the class limit	Normalised overstrain
<code>type_ordinal</code>	Product class as 0, 1, 2	Model input

Dataset quality report

Every dataset, whether bundled or uploaded, is profiled before its predictions are relied on. The report contains:

- rows received, accepted and rejected, with the reason for each rejection
- each signal's observed range compared with the range the model was trained on
- product mix and, where labels are present, the recorded failure rate
- the number of rows already breaching each physical limit

It concludes with a verdict:

Verdict	Condition
Ready	All checks pass
Use with care	Over 5 percent of rows rejected, over 1 percent of a signal outside the training range, fewer than 20 usable rows, or training ranges unavailable
Not usable	No rows survive validation

5. Failure prediction

Model structure

Five binary classifiers share one feature set: one for overall failure risk and one for each modelled failure mode (HDF, PWF, OSF, TWF). Separate models allow each to carry its own decision threshold and let the agent re-score a single failure mode after an adjustment.

Data is split into training (6,400), validation (1,600) and test (2,000) records, stratified on overall failure. Thresholds and model choice use the validation set only; the test set is used solely for reporting.

Automated model selection

For each of the five models, three candidates are trained on identical data: XGBoost, LightGBM and a logistic regression baseline. The winner is the candidate with the highest precision recall AUC (PR AUC) on the validation set, a measure suited to rare events such as failures. Candidates within 0.002 of the best are treated as tied, and ties favour the earlier candidate so the choice is stable between training runs. Only tree based models are eligible, because root cause analysis relies on SHAP's tree explainer; the baseline is measured as a reference point.

Model	Selected	Validation PR AUC (XGBoost / LightGBM / baseline)	Test PR AUC
Overall failure	LightGBM	0.845 / 0.865 / 0.526	0.890
Heat dissipation	LightGBM	0.991 / 1.000 / 0.634	1.000
Power	LightGBM	0.900 / 0.949 / 0.881	0.984
Overstrain	XGBoost	1.000 / 0.997 / 0.995	1.000
Tool wear	XGBoost	0.135 / 0.105 / 0.105	0.066

The tool wear model is weak by construction: the dataset places tool wear failures at a random point within a 200 to 240 minute wear window, so no signal can time them. The system handles this mode through the

physical rule and a maintenance action rather than the model's score.

Experiment tracking

Each candidate is logged to MLflow as a child run of one training run, with its hyperparameters, validation and test metrics, fit time, and whether it was eligible and selected. Tracking uses a local SQLite store and is viewed with `make mlflow`. MLflow is a development dependency; the production image records the same comparison in the model bundle without it.

Remaining useful life estimate

The dataset contains no run to failure history, so remaining useful life (RUL) cannot be learned directly. It is derived from failure probability through a constant hazard model. Treating the probability `p` as the chance of failure within a 72 hour reference horizon `H`:

$$\text{lambda} = -\ln(1 - p) / H \text{ and } \text{RUL} = 1 / \text{lambda}$$

The estimate is capped at 720 hours, beyond which a single operating point carries no meaningful information. Because it decreases strictly as risk rises, any adjustment that lowers risk appears as hours gained. Every response labels RUL as an estimate.

Risk bands

Band	Failure probability
Critical	0.60 and above
High	0.30 to 0.60
Elevated	0.10 to 0.30, or any breached physical limit
Normal	below 0.10 with no breached limit

A breached physical limit always raises an asset to at least Elevated, since the breach is an observed fact while the probability is an estimate.

6. Explainability

SHAP's TreeExplainer attributes each prediction to the input features. For every at risk asset the system reports:

- each feature's contribution and whether it raises or lowers risk
- its share of the total upward contribution, shown as root cause confidence
- the control lever it maps to: torque, speed, ambient conditions, maintenance, or fixed

Aggregating contributions by lever tells the agent which setpoint to adjust. Root cause is reported only for assets that are at risk, so a healthy asset never shows an alarming driver.

7. Prescriptive agent

Agent graph

The agent is a LangGraph state machine of six nodes. Each node is a pure function over shared state, which makes every step independently testable.

Node	Function
Ingest	Loads the selected failure models
Diagnose	Risk, RUL estimate, failure mode, physical margins, SHAP root cause
Simulate	Generates and scores candidate adjustments
Decide	Applies safety guards and selects the action
Narrate	Writes the plain language explanation
Validate	Assembles the response and enforces the output schema

All figures come from the models, the physical rules and the cost model. A language model, when configured, is used only in the Narrate node to phrase the explanation; without one, a deterministic template produces the same fields, so the system runs fully offline and its decisions are reproducible and auditable.

Physical failure rules

Mode	Failure condition
Tool wear (TWF)	Tool wear between 200 and 240 min
Heat dissipation (HDF)	Temperature difference below 8.6 K and speed below 1380 rpm
Power (PWF)	Shaft power outside 3,500 to 9,000 W
Overstrain (OSF)	Wear times torque above 11,000 (L), 12,000 (M) or 13,000 (H) min Nm

These rules give the agent the signed distance to every limit, which determines the direction of a fix. Heat dissipation failure, for example, requires both low temperature difference and low speed, so the correct response is to increase speed rather than reduce load.

Counterfactual search

The agent evaluates 167 candidates:

- 83 setpoint changes combining torque steps (from 20 percent lower to 20 percent higher) with speed steps (from 5 percent lower to 20 percent higher)
- a tool replacement at the next line stop
- the tool replacement paired with each setpoint change, for assets breaching limits that require different kinds of fix

All candidates are scored in a single batched model call.

Safety guards

- 1. **Feasibility.** A candidate is rejected if it would breach a limit the asset currently respects, reduce output by more than 25 percent, raise it by more than 30 percent, or leave the drive's operating range.
- 2. **Fault resolution first.** Candidates that clear every breached limit rank ahead of those that merely lower the risk score, so a low cost action that ignores the actual fault cannot win.
- 3. **Stop rather than overpromise.** When no candidate clears a breached limit, the agent recommends stopping the asset and lists the best partial options as non prescribed alternatives.

Selection

Among candidates that reach the maintenance window, the agent chooses by, in order: least output lost, fewest separate actions, smallest setpoint change, and largest risk reduction. The three next best options are returned as alternatives.

Economic model

- $\text{Avoided cost} = \text{risk reduction} \times (\text{unplanned downtime hours} \times \text{downtime cost per hour} + \text{emergency callout cost})$
- $\text{Throughput cost} = \text{production value per hour} \times \text{output loss} \times \text{hours to the window}$
- $\text{Net benefit} = \text{avoided cost} - \text{throughput cost}$

Defaults are 8 hours of downtime at \$1,800 per hour, a \$2,500 callout and \$950 of production per hour. All four are configurable and returned with every response.

Output contract

Responses conform to a strict Pydantic schema (`PrescriptiveRecommendation` , version 1.0) that rejects unknown fields and bounds every probability. It contains the telemetry snapshot, risk assessment, root cause, action type, adjustments, projection, economics, explanation, alternatives, confidence, caveats and the agent's reasoning steps. The action type is one of `derate` , `reconfigure` , `schedule_maintenance` , `no_action` or `stop_now` .

8. API reference

Interactive documentation is available at `/docs` on any running instance.

Method	Path	Description
GET	<code>/health</code>	Service status and model availability
GET	<code>/model</code>	Model selection results, metrics and thresholds
POST	<code>/datasets</code>	Upload a telemetry CSV (up to 16 MB)

Method	Path	Description
GET	/datasets	List datasets
GET	/datasets/{id}	Dataset summary, risk counts and quality verdict
GET	/datasets/{id}/quality	Dataset quality report
GET	/datasets/{id}/assets	Assets by risk, with band filter and paging
GET	/datasets/{id}/assets/{asset_id}	Risk, margins and root cause for one asset
GET	/datasets/{id}/assets/{asset_id}/history	Telemetry series for charting
POST	/datasets/{id}/assets/{asset_id}/recommendation	Run the agent for a maintenance window
POST	/datasets/{id}/assets/{asset_id}/apply	Record an accepted adjustment and its projected effect
POST	/telemetry/score	Score telemetry rows without storing them
POST	/recommendations	Run the agent on inline telemetry

The apply endpoint records an operator's acceptance and reports the projected operating point. The prototype has no connection to physical drives and states this in every apply response.

9. Dashboard

The React dashboard opens on the highest risk asset, so the first screen shows a complete diagnosis and prescription.

- **Asset selection:** assets ranked by risk, filterable by risk band, searchable by identifier and paged.
- **Asset health:** failure probability, RUL estimate, likely failure mode, live signals and breached limits.
- **Analysis tabs:** SHAP root cause with physical margins, telemetry charts, and the model comparison.
- **Agent reasoning:** each agent step with its findings and duration.
- **Recommendation:** an adjustable maintenance window, a failure timeline comparing failure with and without the change, the exact setpoint changes, projected impact, economics, caveats and alternatives, and an Implement Adjustment action.
- **Dataset workspace:** dataset selection, CSV upload and the quality report.

Light and dark themes are supported, and the layout adapts to narrow screens.

10. Deployment

Container

A multi stage Dockerfile builds the dashboard with Node, then installs the Python runtime, trains the models during the build, and copies in the compiled dashboard. Training at build time means a fresh instance serves immediately, and a model that fails to train fails the build rather than a deployment. The image runs as a non root user and exposes a health endpoint.

Hosting

The service runs on Google Cloud Run, which builds the image from source and serves it over HTTPS.

Setting	Value	Reason
Minimum instances	0	Scales to zero when idle
Maximum instances	1	Uploaded datasets are held in memory
Memory	1 GiB	Measured working set about 200 MB
CPU	1, billed during requests only	Low cost for intermittent use

A registry cleanup policy retains only the latest image.

Configuration

Variable	Default	Purpose
<code>PORT</code>	8000	Listening port, set by Cloud Run
<code>ANTHROPIC_API_KEY</code>	unset	Optional language model for explanations
<code>DSC_SEED_SAMPLE</code>	1	Load the bundled dataset at startup
<code>DSC_CORS_ORIGINS</code>	local development ports	Only needed for callers on another origin
<code>DSC_*_COST_*</code>	see <code>.env.example</code>	Economic model constants
<code>DSC_MLFLOW</code>	1	Enable experiment tracking during training

11. Testing and validation

Automated tests

Area	Tests
Physical rules	8
Data pipeline	6
Quality report	10
Remaining useful life	4
Models and explainability	7
Model selection	8
Counterfactual search	13
Output schema	6
Agent	19
API and dashboard serving	31
Whole pipeline integration	14
Total	126

API tests exercise the real application, models and agent with nothing mocked. The integration suite runs sampled records through the full agent and asserts invariants that must hold for every record: no prescription introduces a new failure, every prescription either clears the breached limit or recommends stopping, projections and economics are internally consistent, every response satisfies the schema, and identical input produces identical output.

Validation across all critical assets

Outcome	Assets
Setpoint reconfiguration	148
Maintenance, including combined maintenance and setpoint fixes	134
Derate	21
Stop, no safe change available	11
Prescriptions introducing a new failure	0
Prescriptions leaving a breached limit in place	0

12. Setup instructions

Prerequisites

- Python 3.12 or newer
- Node.js 20 or newer
- On macOS, the OpenMP runtime: `brew install libomp`

Run locally

```
git clone https://github.com/riajha-usc/drive-saver-copilot.git
cd drive-saver-copilot
make setup      # virtual environment and dependencies
make train      # train and select the models
make test       # run the test suite
make ui-build   # build the dashboard
make api        # dashboard and API on http://127.0.0.1:8000
```

API documentation is served at <http://127.0.0.1:8000/docs>. For dashboard development with live reload, run `make ui` in a second terminal and open <http://127.0.0.1:5173>.

Experiment tracking

```
make mlflow      # MLflow interface on http://127.0.0.1:5000
```

Docker

```
make docker-build
make docker-run   # http://127.0.0.1:8000
```

Deploy to Google Cloud Run

```
gcloud run deploy drive-saver-copilot --source . --region us-central1 \
  --allow-unauthenticated --memory 1Gi --cpu 1 \
  --min-instances 0 --max-instances 1
```

Optional language model

Copy `.env.example` to `.env` and set `ANTHROPIC_API_KEY`. All numbers are identical with or without it; only the wording of explanations changes.

13. Limitations and future work

Current limitations

- **Snapshot data.** The dataset contains independent operating points rather than time series, so RUL is estimated from risk rather than learned, and the telemetry chart shows records in sequence rather than one machine over time.
- **Synthetic failure modes.** The dataset's failures follow defined rules. In production the learned models generalise to plants without explicit rules, while the physics layer continues to act as a safety constraint.
- **Prototype state.** Uploaded datasets are held in memory on a single instance, and there is no authentication.
- **Assumed economics.** Cost figures use configurable plant defaults.
- **Tool timing.** Combined fixes count the tool replacement as immediate, although it occurs at the next line stop; responses state this.

Future work

- Persistent storage in PostgreSQL, enabling multiple instances
- Streaming telemetry from drives via OPC UA or MQTT
- RUL models trained on run to failure data
- Closed loop control: approved setpoints written to the drive with an approval workflow
- One click retraining from the dashboard

14. Attribution

- AI4I 2020 Predictive Maintenance Dataset, UCI Machine Learning Repository, CC BY 4.0.
- Open source libraries: XGBoost, LightGBM, scikit-learn, SHAP, MLflow, LangGraph, FastAPI, Pydantic, pandas, NumPy, React, Vite and Recharts, used under their respective licences.